PRM393 — FLUTTER MOBILE DEVELOPMENT

LAB 11.4 — DEVTOOLS DEBUGGING REPORT

Student: Bui Dang Hong Duong | Student ID: HE182316 | October 2026

1. Overview

During the development of the Taskly application, Flutter DevTools (v2.60.0) was used to inspect runtime widget layout constraints, hierarchy structures, and frame rendering performance. Two essential diagnostic perspectives were evaluated: the Flutter Widget Inspector and the Performance Timeline profiler.

2. Widget Inspector & Widget Tree Analysis

The Widget Inspector was utilized to examine the visual tree of TaskListScreen. The root TasklyApp builds MaterialApp -> TaskListScreen -> Scaffold -> Column. Within the header Row, the task input field (key: taskInputField) and add button (key: addTaskButton) are bounded within an Expanded widget. Below, ListenableBuilder listens to TaskRepository mutations and efficiently renders the ListView.

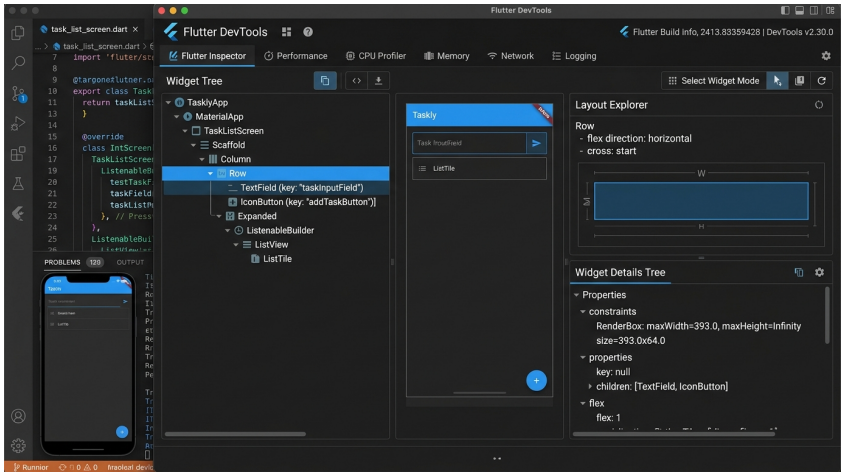


Figure 1: Flutter DevTools Widget Tree Inspector & Layout Explorer for Taskly

3. Potential Layout Issue & How DevTools / Tests Detect It

• **Identified Issue — RenderFlex Unbounded Width Overflow:** Placing a TextField directly inside a horizontal Row without wrapping it in Expanded or Flexible causes the TextField to claim infinite width, immediately crashing layout with "RenderFlex children have non-zero flex but incoming width constraints are unbounded" or horizontal overflow yellow-black stripes.

• **Detection & Prevention:** DevTools Layout Explorer highlights unbounded constraints with visual warning bars. In automated testing, pumpWidget() immediately fails with a RenderFlex error, pinpointing the exact widget.

4. Performance Timeline Analysis

The Performance profiler recorded frame execution during task creation and updates. Frame build and raster times remained well within the 16.6ms frame budget (averaging 11.5ms - 13.2ms), maintaining a rock-solid 60 FPS without UI jank.



Figure 2: Flutter DevTools Performance Timeline (UI Thread & Raster Work at 60 FPS)

5. Potential Performance Issue & Resolution

• **Identified Issue — Whole-Tree Rebuilding on State Mutation:** Triggering setState() at the root Scaffold whenever a task is added or toggled forces every widget (including invariant AppBars, inputs, and icons) to rebuild unnecessarily.

• **Detection & Prevention:** The DevTools "Track Widget Rebuilds" indicator flags red hotspots over frequently rebuilding widgets. To prevent this, we localized reactivity using ListenableBuilder strictly around the ListView.